

POCKETSMART AI

Your Smart Budget & Recommendation Assistant

Project Report

Project Domain	Generative AI • Budget Planning • Recommendation System
Core Technologies	FastAPI • Gemini AI • HTML/CSS • JavaScript • Jinja2
Major Modules	Home Interior • Party Planning • Jewelry Recommendation

Prepared for College Project Submission

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Proposed Solution
6. System Architecture
7. Core Technologies
8. Functional Modules
9. Project Workflow
10. Milestones & Implementation
11. Database, Authentication & Session Management
12. UI/UX Design
13. Testing & Optimization
14. Expected Results
15. Advantages & Limitations
16. Future Enhancements
17. Conclusion

1. Abstract

PocketSmart AI is a Generative-AI-powered budget and recommendation assistant designed to simplify everyday planning. Instead of manually comparing products, services, prices, and platforms, users enter a budget and a small set of preferences. The system interprets the context and generates structured recommendations for home interiors, party planning, and jewelry selection. The Jewelry Planner can additionally use an outfit image as an input for style and color coordination. The application combines a FastAPI backend, Gemini AI, responsive web pages, authentication, session handling, and recommendation history into one modular platform.

The project demonstrates how AI can support practical decision-making while keeping the user's budget as a primary constraint. The system is designed as an MVP with mock or simulated third-party product data, allowing the recommendation workflow to be demonstrated without depending on live commercial APIs.

2. Introduction

Budget planning becomes difficult when users need to consider many categories, price ranges, styles, vendors, and quantities at the same time. A person decorating a room may need lighting, furniture, fans, tables, and décor. Someone organizing a party may need to divide money among food, venue, decoration, and entertainment. Similarly, jewelry selection can depend on occasion, outfit color, style, and budget.

PocketSmart AI addresses these situations through a common recommendation engine. The same platform can adapt its questions according to the selected planner, process the inputs with an AI model, and present the results in an easy-to-understand format.

3. Problem Statement

Users often spend considerable time comparing products and services across different platforms. Price information can vary, choices can be overwhelming, and it is easy to exceed a planned budget. Existing shopping or planning websites generally focus on a single category rather than combining budget allocation, contextual preferences, and AI-assisted recommendations in one workflow.

The proposed system aims to reduce this complexity by accepting a user's budget and context, allocating spending where appropriate, and generating a curated set of recommendations.

4. Objectives

- Build a unified budget-planning platform for multiple lifestyle needs.
- Generate personalized recommendations based on budget, quantity, occasion, and preferences.
- Use Gemini AI for contextual reasoning and multimodal analysis.
- Provide separate planners for home interiors, parties, and jewelry.
- Support optional outfit-image input for jewelry recommendations.
- Maintain user authentication, sessions, recommendation history, and reusable plans.
- Create a modular backend that can later connect to real product/service APIs.

5. Proposed Solution

PocketSmart AI provides a planner-first user experience. After login, the user selects Home, Party, or Jewelry. The application displays only the fields relevant to that scenario. The submitted data is validated by the backend, converted into a structured AI prompt, and sent to Gemini. The response is then formatted into recommendation cards containing item names, estimated prices, reasons for selection, and platform/source information when available.

For an MVP, third-party platform data can be represented through mock datasets or simulated API responses. In a production system, official APIs or approved partner feeds should be used wherever available rather than relying on unauthorized scraping.

6. System Architecture

Layer	Component	Responsibility
Presentation	HTML/CSS/JavaScript + Jinja2	Collects inputs and displays planner results.
Application	FastAPI	Routing, validation, authentication, sessions and API responses.
AI Service	Gemini AI	Interprets budgets, preferences and optional images; generates recommendations.
Data	Database / mock datasets	Stores users, history and simulated product/service information.
External Sources	Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO etc.	Potential sources for product/service information, subject to API/partner access.

High-level data flow: User → Planner Form → FastAPI → Validation → Gemini AI / Recommendation Service → Structured Results → Web UI → History.

7. Core Technologies

Technology	Purpose
Python	Primary backend programming language.
FastAPI	High-performance API framework for routes, validation and backend services.
Gemini AI	Generative AI layer for contextual recommendation generation and image-aware analysis.
HTML/CSS	Responsive and accessible web interface.
JavaScript	Dynamic form behaviour, API requests and result rendering.
Jinja2	Server-side HTML templating for planner pages and authenticated views.
JWT / Session Controls	Authentication and protected access to personalized features.
Uvicorn	ASGI server used to run the FastAPI application.

8. Functional Modules

8.1 Home Interior Budget Planner

The user enters a total budget, room type, required quantities, preferred style, and other constraints. The system can divide the budget across furniture, lighting, fans, décor, and related categories.

Gemini is then prompted to select options that balance price, functionality, and the user's stated style.

Example inputs: Living Room, budget ₹50,000, 4 lights, 1 ceiling fan, 1 dining table, modern style.

8.2 Party Budget Planner

The Party Planner collects event type, guest count, budget, venue details, date or duration, and preferences. The recommendation service can create a budget allocation such as food, decoration, venue, and entertainment. Suggestions may be generated from mock vendor data or authorized external sources.

8.3 Jewelry Budget Planner

The Jewelry Planner accepts budget, occasion, jewelry type, preferred style, and optional outfit image. The AI can use the image as contextual input for broad color/style coordination and return suitable categories such as earrings, necklaces, bracelets, or sets. The system should present image analysis as a recommendation aid rather than an objective fashion judgment.

9. Project Workflow

Step 1: User registration/login

Step 2: Select a planner

Step 3: Enter budget and contextual preferences

Step 4: Upload an optional outfit image for Jewelry Planner

Step 5: FastAPI validates and structures the request

Step 6: Recommendation service prepares a domain-specific prompt

Step 7: Gemini generates structured recommendations

Step 8: Backend formats the response and applies budget checks

Step 9: Frontend displays recommendation cards

Step 10: Query and results are saved to recommendation history

10. Milestones & Implementation

Milestone 1 - Gemini AI Initialization

- Create the required Google Cloud / Gemini access and configure API credentials securely.
- Store secrets in environment variables rather than hard-coding API keys.
- Test a text-only request and a multimodal request with an approved sample image.
- Design initial prompts for budget interpretation and structured output.

Milestone 2 - Core Functionality Development

- Initialize FastAPI and required libraries.
- Create reusable recommendation functions and domain-specific prompt templates.
- Implement `/generate-home`, `/generate-party`, and `/generate-jewelry`.
- Add validation, error handling, and fallback recommendations.

Milestone 3 - Backend Integration

- Implement /register, /login, /logout and /token.
- Implement /session-info and /session-data for personalized workflows.
- Implement /recommendations-details and /history.
- Configure CORS carefully for the frontend origin.
- Use startup logic for configuration and service initialization.

Milestone 4 - UI Development

- Create landing page, registration, login and dashboard.
- Create separate planner forms for Home, Party and Jewelry.
- Render AI output as clean recommendation cards.
- Provide a history page for previous plans.

Milestone 5 - Testing & Optimization

- Test multiple budgets, quantities, guest counts and preferences.
- Check whether recommendations remain within the user's budget.
- Test missing, invalid and extreme inputs.
- Test image upload failures and unsupported formats.
- Improve prompts and add fallback results when AI output is incomplete.

11. Database, Authentication & Session Management

The platform requires secure user management because recommendation history and saved plans are personalized. A typical implementation stores a user identifier, securely hashed password, creation timestamp, and relevant preference data. Passwords must never be stored in plain text.

JWT-based authentication can issue an access token after successful login. Protected routes should verify the token before returning personalized data. Session endpoints such as /session-info and /session-data can provide limited metadata required by the frontend. Logout should invalidate or expire the relevant session/token strategy used by the application.

Recommended data entities include: User, RecommendationHistory, PlannerRequest, RecommendationItem, and SavedPlan.

12. UI/UX Design

- Home Page – introduces PocketSmart AI and provides navigation to the planners.
- Register Page – creates a new account.
- Login Page – authenticates an existing user.
- Dashboard – shows recent recommendations and saved activity.
- Home Interior Planner – collects room, quantity, budget and style information.
- Home Recommendations – presents categorized products and estimated prices.
- Party Planner – collects event type, guest count, budget and venue requirements.
- Party Recommendations – presents food, venue, decoration and entertainment suggestions.
- Jewelry Planner – collects occasion, budget and optional outfit image.
- Jewelry Recommendations – displays matching categories, estimated price and rationale.
- History Page – shows previous recommendation requests and results.

13. Testing & Optimization

Test Area	Sample Check	Expected Result
Budget validation	Budget = ₹20,000; selected items exceed budget	System flags or replaces expensive items.
Home planner	Room + quantities + style	Categorized home recommendations are returned.
Party planner	Guest count + event + budget	Budget allocation and party options are generated.
Jewelry planner	Occasion + budget + image	Style-aware jewelry suggestions are returned.
Invalid input	Negative budget / missing required field	Clear validation message is shown.
Authentication	Protected endpoint without valid token	Access is rejected.
AI failure	Model/API unavailable	Fallback message or default dataset is shown.

Test Area	Sample Check	Expected Result
Responsive UI	Mobile and desktop screens	Forms and recommendation cards remain usable.

Optimization priorities include structured JSON output, stronger budget validation, retry/fallback logic, caching where appropriate, prompt versioning, and consistent product-data schemas.

14. Expected Results

- A single web application capable of handling three different budget-planning scenarios.
- Faster discovery of suitable products and services through AI-assisted filtering.
- Recommendations that consider both budget and contextual preferences.
- An optional multimodal jewelry workflow using outfit images.
- Secure personalized access with history and reusable recommendations.
- A modular foundation that can later connect to live commerce and service APIs.

15. Advantages & Limitations

Advantages

- One platform for multiple lifestyle planning categories.
- Budget-first recommendation logic.
- Personalized inputs rather than generic lists.
- Optional image-aware recommendations for jewelry.
- Modular FastAPI architecture supports future expansion.
- History and authentication enable a personalized experience.

Limitations

- AI-generated prices may become outdated unless backed by current data sources.
- Commercial platforms may restrict automated access; official APIs or approved feeds should be used.
- AI recommendations can contain errors or incomplete product information and therefore require validation.
- Image-based style analysis should be treated as an assistive feature, not a definitive judgment.
- Live availability, delivery charges, taxes and location-specific pricing may change.

16. Future Enhancements

- Integrate authorized real-time product and service APIs.
- Add location-aware recommendations and delivery/availability checks.
- Introduce price comparison and price-drop alerts.
- Allow users to save and edit complete budget plans.
- Add multilingual support, including Tamil and other Indian languages.
- Provide explainable budget allocation charts and downloadable plans.
- Add admin analytics for anonymized system performance monitoring.
- Use feedback loops to improve recommendation quality over time.

17. Conclusion

PocketSmart AI demonstrates a practical application of Generative AI for budget-aware lifestyle planning. By combining FastAPI, Gemini AI, responsive web technologies, authentication, and modular recommendation services, the system turns a simple budget into a structured planning experience. Its three core modules—Home Interior, Party Planning, and Jewelry Recommendation—show how the same AI architecture can adapt to different real-world contexts.

The project is designed as an extensible MVP. With reliable data integrations, stronger validation, secure deployment, and continuous testing, PocketSmart AI can evolve from a college prototype into a broader recommendation platform. The key value of the system is not simply generating suggestions, but organizing those suggestions around the user's budget, context, and preferences in a clear and usable way.

End of Project Report